

NEXA — Contratos de Dados e Interfaces do Sistema (v1)

1. Objetivo

Definir, de forma formal e padronizada:

- estrutura dos dados trafegados entre camadas
- contratos entre frontend, domínio e infraestrutura
- formatos de entrada e saída (DTOs)
- regras de transformação (mappers)

Este documento garante:

- consistência de dados
 - desacoplamento entre camadas
 - previsibilidade na implementação
 - base sólida para evolução futura (API, mobile, integrações)
-

2. Princípio Arquitetural

O sistema deve operar com separação clara entre:

1. **Banco de dados (estrutura física)**
2. **DTOs (estrutura de transporte)**
3. **Entidades de domínio (regra de negócio)**
4. **View Models (estrutura de interface)**

Regra obrigatória:

Nenhuma camada deve depender diretamente da estrutura da outra.

3. Estrutura de Camadas

3.1 Database (Supabase / PostgreSQL)

Representa o dado bruto persistido.

Exemplo:

```
type NegotiationRow = {  
  id: string  
  account_id: string  
  development_id: string  
  unit_id: string  
  client_id: string  
  broker_id: string  
  status: string  
  created_at: string  
  updated_at: string  
}
```

3.2 DTO (Data Transfer Object)

Formato de transporte entre infraestrutura e aplicação.

```
type NegotiationDTO = {  
  id: string  
  unitId: string  
  clientId: string  
  brokerId: string  
  status: NegotiationStatus  
  createdAt: Date  
  updatedAt: Date  
}
```

Regras:

- nomes em camelCase
- datas já convertidas para Date
- enums já aplicados

3.3 Entidade de Domínio

Representa o objeto com comportamento e regras.

```
class Negotiation {
  constructor(
    public id: string,
    public unitId: string,
    public clientId: string,
    public brokerId: string,
    public status: NegotiationStatus
  ) {}

  canSendProposal(): boolean {
    return this.status === "IN_PROGRESS"
  }

  approve() {
    if (this.status !== "UNDER_ANALYSIS") {
      throw new Error("Invalid state transition")
    }
    this.status = "APPROVED"
  }
}
```

Regras:

- contém lógica de negócio
- nunca acessa banco diretamente
- nunca depende de React

3.4 View Model (UI)

Formato usado pela interface.

```
type NegotiationView = {
  id: string
  clientName: string
  unitLabel: string
  statusLabel: string
  statusColor: string
}
```

Regras:

- preparado para exibição
 - pode agregar dados
 - não contém regra de negócio
-

4. Mappers (Transformação de Dados)

4.1 Database → DTO

```
function mapNegotiationRowToDTO(row: NegotiationRow): NegotiationDTO {
  return {
    id: row.id,
    unitId: row.unit_id,
    clientId: row.client_id,
    brokerId: row.broker_id,
    status: row.status as NegotiationStatus,
    createdAt: new Date(row.created_at),
    updatedAt: new Date(row.updated_at)
  }
}
```

4.2 DTO → Domínio

```
function mapDTOToNegotiation(dto: NegotiationDTO): Negotiation {
  return new Negotiation(
    dto.id,
    dto.unitId,
    dto.clientId,
    dto.brokerId,
    dto.status
  )
}
```

4.3 Domínio → View

```
function mapNegotiationToView(
  negotiation: Negotiation,
  clientName: string,
  unitLabel: string
): NegotiationView {
  return {
```

```
id: negotiation.id,  
clientName,  
unitLabel,  
statusLabel: getStatusLabel(negotiation.status),  
statusColor: getStatusColor(negotiation.status)  
}  
}
```

5. Contratos por Entidade

5.1 Negotiation

DTO:

```
type NegotiationDTO = {  
  id: string  
  unitId: string  
  clientId: string  
  brokerId: string  
  status: NegotiationStatus  
  createdAt: Date  
  updatedAt: Date  
}
```

5.2 Proposal

```
type ProposalDTO = {  
  id: string  
  negotiationId: string  
  totalValue: number  
  entryValue: number  
  installments: number  
  status: ProposalStatus  
  createdAt: Date  
}
```

5.3 Reservation

```
type ReservationDTO = {  
  id: string  
  unitId: string  
  negotiationId: string  
  expiresAt: Date  
  status: ReservationStatus  
}
```

5.4 Unit

```
type UnitDTO = {  
  id: string  
  developmentId: string  
  label: string  
  price: number  
  status: UnitStatus  
}
```

5.5 Client

```
type ClientDTO = {  
  id: string  
  name: string  
  document: string  
  phone: string  
}
```

5.6 Broker

```
type BrokerDTO = {  
  id: string  
  name: string  
  brokerageId?: string  
}
```

6. Repositórios (Interface)

Define como a aplicação acessa dados.

```
interface NegotiationRepository {  
  findById(id: string): Promise<Negotiation>  
  create(data: CreateNegotiationInput): Promise<Negotiation>  
  updateStatus(id: string, status: NegotiationStatus): Promise<void>  
}
```

Regra:

- retorna entidade de domínio
- nunca retorna row direto

7. Inputs de Criação

7.1 Criar Negociação

```
type CreateNegotiationInput = {  
  unitId: string  
  clientId: string  
  brokerId: string  
}
```

7.2 Criar Proposta

```
type CreateProposalInput = {  
  negotiationId: string  
  totalValue: number  
  entryValue: number  
  installments: number  
}
```

8. Regras Críticas

1. Nenhuma tela acessa diretamente o banco
 2. Nenhum componente React contém regra de negócio
 3. Nenhuma entidade conhece estrutura de banco
 4. Toda transformação passa por mapper
 5. Tipos são centralizados e reutilizáveis
-

9. Estrutura de Pastas Recomendada

```
domain/  
  entities/  
  enums/  
  services/
```

```
infrastructure/  
  repositories/
```

mappers/
supabase/

modules/
negotiations/
dto/
view/
hooks/

10. Critério de Validação

Este documento está corretamente aplicado quando:

- nenhuma tela usa dados brutos do banco
 - não existe duplicação de tipos
 - mudanças no banco não quebram a UI
 - domínio mantém consistência independente da interface
-

11. Síntese

Este documento estabelece a base de comunicação do sistema.

Sem essa camada:

- o sistema se torna acoplado
- a evolução fica limitada
- o risco de inconsistência aumenta

Com essa camada:

- o sistema se torna previsível
- a implementação acelera
- a escalabilidade é viabilizada